Achieving Mediocrity in a Solitary Domain Through World Models

Xavier O'Keefe*
*Ann & H.J. Smead Department of Aerospace Engineering Sciences, University of Colorado, Boulder, Boulder, CO, USA
xavier.okeefe@colorado.edu

***Abstract*—This work documents the challenges of adapting DreamerV3, a state-of-the-art model-based reinforcement learning algorithm, for robotic manipulation in simulation. While DreamerV3 demonstrates impressive benchmark performance, deploying it on a novel task requires extensive hyperparameter tuning and debugging. Through systematic experimentation with a simulated Franka Emika Panda Arm performing a reaching task, this work identifies key failure modes and engineering challenges that arise when moving from standard benchmarks to custom robotic applications. The results provide practical insights for researchers attempting similar deployments and highlight the gap between algorithmic promise and practical implementation.**

I. Introduction

While reinforcement learning has shown immense promise in the field of robotics, several challenges plague modern algorithms. One of the most critical is sample efficiency. Model-free methods, which learn policies directly from trial-and-error, require vast numbers of environment interactions, making real-world deployment expensive. Many researchers address this problem with simulation, where they can run many parallel trajectories simultaneously in a simulated world. But, this has issues transferring to real world robots, where differences in movement characteristics, textures, and lighting can cause these sim-trained models to completely fail. This is called the sim2real gap.

World models offer an alternative approach that enables sample-efficient learning directly in the desired deployment domain. The Dreamer algorithm learns a compact model of environment dynamics from limited interactions, which it then uses to imagine trajectories in a latent space. This separates dynamics learning from policy training. As a result, the world model requires only modest real-world data, while the policy can be efficiently trained for thousands of gradient steps on these imagined rollouts without requiring additional physical samples. Both components adapt simultaneously to the same environment, ensuring the learned behaviors transfer directly to the system without domain gap issues.

This paper documents the adaptation of Dreamer V3 for robotic manipulation, detailing the technical infrastructure required and the challenges encountered in achieving stable learning. While the agent did not converge to a successful policy within the project timeline, this work provides insights into the practical considerations and failure modes when adapting state-of-the-art world models to real-time robotic control.

II. Background and Related Work

The Dreamer series of algorithms has captivated the attention of the RL community since its introduction in 2019 [1], when it achieved state of the art performance in MuJoCo tasks with a model-based approach. This showed that world models could be used in long-horizon tasks in a computationally efficient manner, contrarily to other model-based approaches like MCTS. Development of the Dreamer algorithm was continued, peaking in 2022 with the third version [2]. This algorithm was able to consistently find diamonds in Minecraft, and will be the focus of this work. Minecraft was seen as a great RL challenge because it requires long-range planning in a procedurally generated environment with very sparse rewards. While other algorithms like VPT [3] had found diamonds, they required thousands of hours of training on human data, as well as hundreds of industry grade GPUs. Dreamer shattered the community's expectations by consistently finding diamonds without any human data and with a single A100 GPU.

Dreamer's applicability to robotics was demonstrated in 2022 with DayDreamer [4], which was able to train robots based on limited amounts of strictly real-world data. Although this paper was released before Dreamer V3, the algorithm used was very close to the final version of Dreamer V3 [5]. The release of Dreamer V4 [6] in 2025 contained significant architecture changes that, while helpful for large scale applications, made it less suitable for small-scale robotics. There have also been several interesting world models released in 2025 [7] [8], but Dreamer V3 was selected for this work due to its proven robotics applicability, open-source code, and stable architecture for resource-constrained settings.

A. Dreamer V3 Methods

Dreamer's architecture comprises three core components that are trained simultaneously: a world model, an actor, and a critic. The world model learns a compact representation of the environment dynamics in a lower-dimensional latent space through direct interaction with the environment. Once trained, this world model enables the agent to simulate imagined trajectories entirely in latent space, which are then used to train the actor and critic networks. This separation between learning the world model from real experience and training behaviors through imagination means the world model requires relatively few environment interactions to learn accurate dynamics, while the actor and critic can be updated many times using imagined rollouts. This approach significantly improves sample

efficiency, enabling the algorithm to learn high-quality policies with substantially fewer real-world samples than model-free methods.

1) World Model: Dreamer's world model is implemented as a Recurrent State Space Model (RSSM), which maintains both deterministic and stochastic components to capture environment dynamics. The RSSM contains several interconnected components:

- **Sequence Model:** A recurrent neural network (GRU) that maintains a deterministic hidden state by processing the previous hidden state, stochastic state, and action: $h_t = f_\phi(h_{t-1}, z_{t-1}, a_{t-1})$
- **Encoder:** Maps sensory inputs x_t and the deterministic state h_t to stochastic latent representations $z_t \sim q_\phi(z_t|h_t, x_t)$, capturing the observed state of the environment
- **Dynamics Predictor:** Predicts the next stochastic state $\hat{z}_t \sim p_\phi(\hat{z}_t|h_t)$ based solely on the deterministic state, enabling imagination without requiring actual observations

In addition to these core components, the world model includes a decoder that reconstructs observations $\hat{x}_t$ from the latent state, a reward predictor that estimates expected rewards, and a continue predictor that predicts episode termination. Together, these components enable the world model to both capture environment dynamics from experience and generate imagined trajectories for policy learning.

B. Actor & Critic

Dreamer employs an actor-critic architecture where the actor network proposes actions and the critic network estimates their long-term value. Both networks are trained entirely within the learned latent space using imagined trajectories generated by the world model, rather than requiring real environment interactions. This allows for orders of magnitude more updates than would be feasible with real samples, while the world model itself is updated only when new environment data is collected. The actor is trained to maximize expected returns estimated by the critic, while the critic learns to predict cumulative rewards through temporal difference learning on imagined rollouts.

C. Robotics

The Dreamer series of algorithms can also be used on robotics applications. DayDreamer [4] shows that a minimally modified Dreamer algorithm can learn complex policies in robotics. Creating a generalized algorithm for robotics has been an open challenge, and the world model structure of Dreamer provides some interesting benefits. Firstly, the world model is effectively a learned simulator, which allows developers to bypass the expensive and time-consuming act of developing one. This also allows the robot to bypass the sim2real gap by never training in sim - Dreamer learns its own world models and learns to execute policies in that world model, which is by default learned to be as close to the real environment as possible. Dreamer is also able to generalize to a very diverse range of input, output, and reward spaces. This means that Dreamer *should* work across a diverse domain of robotic embodiments and tasks. The authors claim that this can be done with a universal set of hyperparameters, and with no modification to the code.

III. Problem Formulation

The project's initial goal was to train the Panda arm to perform a pick and place task using proprioceptive inputs. After significant instability and infrastructure shortcomings, the task was simplified to a end effector reaching task to isolate simulation and algorithmic failures.

A. The Reaching Task

The arm will start in a randomized position in a predefined zone, centered on the goal point. The agent's task is to minimize the Euclidean distance between its end effector and the goal point.

B. Input and Output Space

The model will take inputs from several ROS topics. Those are:

Topic	Description
arm_joints	Vector of joint angles [rad]
current_ee_pose	Pose

And the model will output a 3 dimensional vector of action deltas:

$$a = [\,\Delta x,\ \Delta y,\ \Delta z].$$

IV. Solution Approach

This project will use the official open-source JAX implementation of Dreamer V3 [9]. Minimal modifications were made to the source code, but substantial amounts of infrastructure were required to be built.

A. Arm Setup

The environment uses a simulated Franka Emika Panda Arm and Unity for graphics and physics. Unity is connected to ROS, which facilitates data flow between Unity and downstream applications. ROS also handles control inputs and motion planning for the arm. The ROS connection enables the algorithm to interact with the simulation in real time, receiving data inputs and outputting control signals to the robot. A substantial amount of time spent on this project was spent debugging the simulator and the connection to Dreamer.

The implementation of Dreamer used in this project requires Python 3.10.12 with specific package versions. This Python version, and many of the packages, were incompatible with the installed ROS version. In order to avoid debugging dependency issues, a virtual environment was used to run Dreamer, and a standalone ZeroMQ IPC "bridge" node was used to communicate between this environment and the system ROS network. See Figure 1.

Fig. 1: System diagram showing the data flow between the simulator, ROS2, the bridge node, and the Dreamer agent.

B. Dreamer Setup

Dreamer requires several components to be configured correctly before it can be trained. The first is an gym-style environment file, which sets up inputs, outputs, workspace boundaries, and rewards. The workspace bounds were chosen to prevent the arm from hitting objects in its surroundings (Table I).

TABLE I: Workspace Bounds

Action	Min	Max
x	-0.2	0.2
y	0.15	0.5
z	0.25	0.55

This reward was structured such that the completion bonus would always result in a positive reward, which incentivizes the agent to move in all scenarios. The reward function is defined below, with $\mathbf{x}_t$ being the current state, and $\mathbf{x}^\star$ being the goal state. ϵ was 0.03 meters for the results presented in this paper.

$$r_t = -3\|\mathbf{x}_t - \mathbf{x}^\star\|_2 + 50\,\mathbb{I}(\|\mathbf{x}_t - \mathbf{x}^\star\|_2 < \epsilon)\,.$$

A configuration file is also required, specifying model parameters like size, entropy, batch size, episode length, etc. Dreamer comes with a default set of parameters, intended for higher-complexity tasks. The final configuration used for the reaching task is provided in Table II. As detailed in the Adaptations section below, arriving at these values—particularly the `run.train_ratio` and the model sizing—required extensive trial-and-error and re-tuning to stabilize learning for the single-environment setup.

TABLE II: Updated Dreamer Training Parameters

Parameter Key	Custom Value	Default Value
`batch_size`	32	16
`batch_length`	32	64
`jax.prealloc`	`False`	`True`
`run.train_ratio`	2	32.0
`run.envs`	1	16
`run.eval_envs`	1	4
`agent.opt.lr`	1e-4	4e-5
`agent.opt.eps`	1e-6	1e-20
`agent.dyn.rssm.deter`	512	8192
`agent.dyn.rssm.hidden`	512	1024
`agent.dyn.rssm.classes`	32	64
`agent.imag_length`	15	333 *
`agent.loss_scales.repval`	0.5	0.3
`agent.imag_loss.actent`	0.4	3e-4
`agent.slowvalue.rate`	0.01	0.02
`replay.size`	500000	5e6

* Note: Default value is `agent.horizon`, which is 333.

C. Adaptations

The development process followed a cycle of training, analysis, and debugging. Since the simulation infrastructure was custom built, several iterations were required to squash all bugs and ensure proper functionality.

1) Debugging via Training: Initial training runs served as diagnostic tools. While the code appeared to be correct, observing the agent's behavior revealed some underlying logic and synchronization bugs in the ROS-Unity bridge. Through several cycles of identifying and fixing bugs, a working simulator was finally produced.

2) Entropy: Actor entropy was an important parameter to tune. This parameter controls the tradeoff between exploration and exploitation in the policy. The Dreamer V3 default configuration has a very low actor entropy value of 0.0003, which is optimized for highly parallelized environments. This value resulted in stagnant policies that did not move the arm significantly, greatly limiting the arm's ability to explore. DayDreamer used much a much higher entropy value of 0.4, which allowed for more motion from the arm. Higher entropy values are especially important in singular environments, because the agent may not receive high rewards enough to incentivize it away from local extrema. The minimum and maximum entropy parameters were also exposed. These were used to prevent the agent from converging to a local low-effort policy or outputting actions too far from the policy.

3) Optimizing Episode Structure: Initial training runs were conducted with 1000-step long episodes. This resulted in a replay buffer full of near useless states, especially during the early stages of training. By decreasing the amount of steps to 200, the agent had more opportunities to try different trajectories, and was able to encounter the reward more often. This boosted policy learning speed significantly.

Another optimization was to add a termination condition into the environment. Previous trajectories were all the same length with rewards sprinkled randomly throughout. Adding the termination condition helped the policy to understand the task structure and guide the robot towards the goal position. The termination condition also had the effect of speeding up training, as episodes where the agent had found the reward no longer needed to last the full duration.

4) Randomizing Start Positions: In order to increase generalization, the start position of the end effector was randomized within a region. This allowed for a wider distribution of trajectories and learning, resulting in a more robust policy. Each position component is randomized, where $x^\star$ is the center of the reward region.

$$\mathbf{p}' = \begin{pmatrix} x^\star + \Delta x \\ y^\star + \Delta y \\ z^\star + \Delta z \end{pmatrix}, \quad \text{where } \Delta i \sim \mathcal{U}(-\text{offset}, +\text{offset}) \text{ for } i \in \{x, y, z\}$$

This allowed for start points in and around the goal region to be sampled during the run, providing a constant source of reward and grounding the agent.

5) Train Ratio: Train ratio was the single most impactful parameter governing training stability. This parameter controls the number of gradient updates performed on the policy and world model for each single step collected in the real environment. The base Dreamer configuration sets this ratio high (typically 32-1024) for their benchmark experiments, which rely on collecting massive amounts of data in parallel. However, with only a single robot collecting data, this high ratio causes severe overfitting - the training loop exhausts the limited replay buffer by performing far too many updates on the same small pool of experiences. Lowering the train ratio resulted in much more stable experiments that produced positive results. In this experiment, a train ratio in the range of 2-4 was found to provide stable world model and policy improvements.

6) Curriculum Learning: Curriculum learning was used to "teach" the arm where the reward was. The arm started out just outside the goal region, until it could consistently find the target position. The radius was then expanded to .08 and .12 meters from the center of the goal region. This allowed the algorithm to reach the goal state quickly and more often, which sped up training runs.

V. Results

The results of this work were modest. In addition to the numerous setbacks on the architecture side, there were numerous configurations to experiment with. Commonly, training runs would last 1.5 hours before showing signs of failing, so this process was time-intensive. Results shared here were trained for 130,000 steps, over a duration of 11 hours.

A. World Model

The world model was often able to learn quite well, with loss values frequently approaching zero. The speed at which they did this was highly dependent on the train ratio, with runs of ratio 64 converging to 0 in 20,000 steps (with likely overfitting), and runs of ratio 2 converging in about 100,000 steps.

Figure 2 shows an open loop prediction compared with an actual trajectory for the two input topics: actual position, and joint positions. The actual pose input contained quaternions, which have less trivial dynamics than positions. Note that the estimates of position are more accurate than estimates of orientation. Also note the inaccuracies in the arm joints plot. These are due to the non-deterministic motion of the arm as a result of the RRT sampling based motion planner. The instantaneous impact of this can be seen in the center plot of Figure 3.

B. Task Success

Task success was also modest. In order to gauge the model's learning, its performance was benchmarked against a completely random policy. While it does not outclass the random policy, it does perform empirically better across a large number of simulations. See results in Table III.

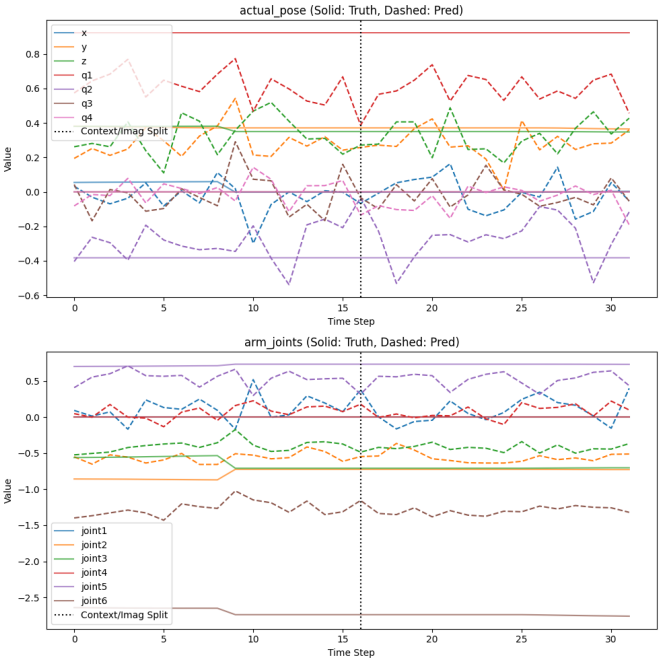

Fig. 2: Plot of predicted vs. actual trajectories for the actual position of the end effector, where (x, y, z) is position in meters and (q1, q2, q3, q4) is the orientation in quaternion form. Arm_joints shows the angles of the arm joints in radians.

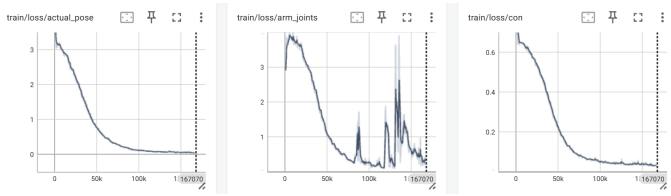

Fig. 3: Training losses over 167,000 steps showing convergence of world model components. The actual_pose loss (left) and continue predictor loss (right) converge smoothly to near-zero values, indicating successful world model learning. The arm_joints loss (center) shows this motion planner-induced non-determinism. In all figures, we see training stability emerge around 100,000 steps. Evaluation of the model was performed after the first spike in arm_joints and before the second.

VI. Discussion

While world models show promise for robotics, this work shows that their practical deployment remains far from straightforward. The path from initial implementation to a functional system is long and involves many turns.

A. Task Design

This work began with an ambitious objective to pick up a block and raise it off the ground, purely from proprioceptive inputs. After initial struggles, the task was simplified to basic reaching to facilitate debugging. The simple nature of this task allowed simulation failures to be easily distinguished

TABLE III: Agent Performance Analysis Results

Agent	Total Runs	Invalid	Incomplete	Completion (%)
Trained	50	9	28	31.7 %
Random	50	5	33	26.7 %

TABLE IV: In the above table, invalid is the amount of times the agent started inside the goal region. Incomplete is the amount of times the agent did not finish the task. The trained agent showed moderate improvement over a random policy.

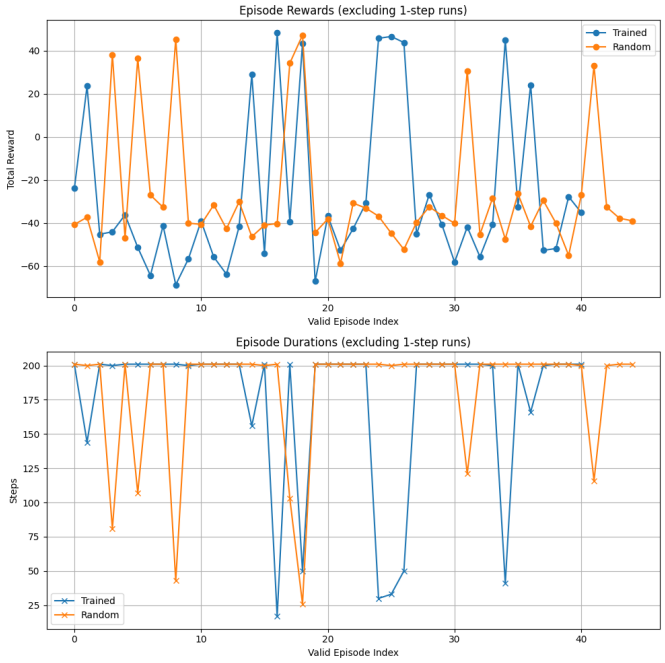

Fig. 4: Plots of the rewards received and episode lengths for the trained vs. untrained agents. Note that, in the reward plots, the lowest scores are exclusively trained agents. These agents attempted to find the goal, but were mislead about the direction. The random agent appears to have stayed roughly stagnant, collecting more reward. Also note the amount episodes that *did not* last 200 steps. Specifically, the trained episodes lasting between 125 and 200 indicate an agent that started far away from the reward, and traveled towards it. This does not happen for the untrained agent.

from algorithmic issues, and the effects of parameter changes could be more clearly observed. While the final reaching task was considerably simpler than the original goal, it provided the diagnostic clarity necessary to evaluate whether the world model could learn meaningful representations at all.

B. Hyperparameter Sensitivity and Model Scale

The default DreamerV3 configuration uses a single set of hyperparameters across all reported benchmark tasks, creating a misleading impression that extensive hyperparameter tuning could be avoided. In reality, those parameters were optimized for large-scale parallelized simulations collecting data orders of magnitude faster than a single real robot. Attempting to apply these same configurations to single-robot deployment resulted in severe overfitting and training instability.

The DayDreamer parameters, designed specifically for single real-robot deployment, proved far more effective as a starting point. Key modifications included reducing the train ratio to 2-4 to match the slower data collection rate, and scaling the RSSM to 512/512/32/32 to better match model capacity to task complexity. These adjustments resulted in stable world model learning and steady policy improvement. However, arriving at these parameters required extensive trial and error.

C. Real World RL

Despite extensive debugging, hyperparameter tuning, and task simplification, the final system achieved very limited success. The agent saw low success rates (Table III), and did not perform much better than a random agent.

This negative result, while disappointing, reveals something about the state of these algorithms. Even with state-of-the-art algorithms, non-zero domain knowledge, and careful engineering, these models do not just work "out of the box".

Low success rates are also a part of the field - many top of the line algorithms struggle to achieve 90% success rates in environments optimized for publish-ability. While this implementation could undoubtedly be improved, the results suggest that DreamerV3's performance for this particular task configuration may plateau significantly below perfect performance. This environment was not the real world, but it was further from the idealized Gym benchmarks than a real robot. These sorts of environments are hard, and low success rates can be expected.

VII. Conclusions and Future Work

World models still show immense promise for robotics. However, the gap between impressive benchmark results and practical deployment remains significant. Bridging it will require not only algorithmic advances, but also better deployment guidelines and improved debugging tools.

Future work will continue on this platform, extending the input space to include visual inputs, and expanding the task definition back to the original pick-and-place. The results suggest that Dreamer V3 is promising, but that more work in the areas of hyperparameter tuning, model sizing, and simulation infrastructure are needed before significant results are achieved.

VIII. Contributions and Release

This work was entirely carried out by Xavier. This work adapts an off-the-shelf Dreamer V3 implementation. The simulation is custom, and entirely the work of Xavier. The authors grant permission for this report to be posted publicly.

A. AI Acknowledgment

The development of this project was significantly boosted by LLMs, specifically Claude and Gemini. There was an incredible amount of boilerplate infrastructure to write and

debug, which the LLMs were massively helpful for. However, the general architecture and solution approach are my own.

B. Code

Code can be found in public GitHub repositories. My fork of the Dreamer repo is: https://github.com/xavier2933/dreamerv3. Code for the simulation can be found here: https://github.com/xavier2933/thesis

References

[1] D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi, "Dream to control: Learning behaviors by latent imagination," *arXiv preprint arXiv:1912.01603*, 2019.

[2] D. Hafner, J. Pasukonis, J. Ba, and T. Lillicrap, "Mastering diverse domains through world models," *arXiv preprint arXiv:2301.04104*, 2023.

[3] B. Baker, I. Akkaya, P. Zhokov, J. Huizinga, J. Tang, A. Ecoffet, B. Houghton, R. Sampedro, and J. Clune, "Video pretraining (vpt): Learning to act by watching unlabeled online videos," *Advances in Neural Information Processing Systems*, vol. 35, pp. 24639–24654, 2022.

[4] P. Wu, A. Escontrela, D. Hafner, P. Abbeel, and K. Goldberg, "Daydreamer: World models for physical robot learning," in *Conference on robot learning*, pp. 2226–2240, PMLR, 2023.

[5] D. Hafner and R. Curran, "Danijar hafner on dreamer, world models, and general intelligence." TalkRL Podcast, 2023. Accessed: 2024-12-04.

[6] D. Hafner, W. Yan, and T. Lillicrap, "Training agents inside of scalable world models," *arXiv preprint arXiv:2509.24527*, 2025.

[7] F. Zhu, H. Wu, S. Guo, Y. Liu, C. Cheang, and T. Kong, "Irasim: A fine-grained world model for robot manipulation," in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9834–9844, 2025.

[8] Y. Wang, R. Yu, S. Wan, L. Gan, and D.-C. Zhan, "Founder: Grounding foundation models in world models for open-ended embodied decision making," in *Forty-second International Conference on Machine Learning*, 2025.

[9] D. Hafner, J. Pašukonis, J. Ba, and T. Lillicrap, "danijar/dreamerv3: Mastering diverse domains through world models." GitHub, 2023.

Appendix A
Reference Descriptions

- [1] "Dream to Control: Learning Behaviors by Latent Imagination": Original Dreamer paper.
- [2] "Mastering Diverse Domains Through World Models": Dreamer V3 paper. The basis of this paper.
- [3] "Video Pretraining (VPT): Learning to Act by Watching Unlabeled Online Videos": VPT paper, which found diamonds in Minecraft by learning from human demonstration videos. Provided a positive comparison for Dreamer V3 when it came out.
- [4] "DayDreamer: World Models for Physical Robot Learning": Showed the application of Dreamer to robots. Provided useful configurations and methodologies to aid the development of this project.
- [5] "Danijar Hafner on Dreamer, World Models, and General Intelligence": Podcast featuring Danijar. Explains differences between Dreamer V3 and DayDreamer.
- [6] "Training Agents Inside of Scalable World Models": Dreamer V4 paper. This algorithm expands the world model framework to larger scales, but has a much different architecture than the other 3 Dreamers.
- [7] "IRASim: A Fine-Grained World Model for Robot Manipulation": Uses world models to learn fine-grained control for robotic arms. Incorporates transformer architecture. I would like to read this at a deeper level.
- [8] "FoUnder: Grounding Foundation Models in World Models for Open-Ended Embodied Decisions": Really cool work combining world models and foundation models. Takes another step towards a general algorithm for robotics tasks.
- [9] "dreamerv3": GitHub repository with the code for Dreamer. This is what I based my implementation on.

Appendix B
Computational Details

The majority of this work was done on my laptop. During the final 3 days, I set up a computer in my lab. This doubled the amount of training runs I could perform over the final few days, which was quite useful.

TABLE V: Compute Platforms

Computer	CPU	GPU	VRAM
Laptop	Intel i9-11900H	NVidia RTX 3050 Ti Laptop	4GB
Desktop	AMD Ryzen 7 3800X	NVidia RTX 2060	6GB